AURA: An End-to-End Multi-Agent Reinforcement Learning Framework for Coordinated Autoscaling of Kubernetes Microservices

Eric T. K.*, Navaneet Krishnan R. V.*, Pranav R. Mallia*
*Department of Computer Science and Engineering
[Your Institution], [City, Country]
{your.email1, your.email2, your.email3}@institution.edu

***Abstract*—Kubernetes Horizontal Pod Autoscaler (HPA) relies on reactive, single-metric threshold policies that consistently underperform under highly dynamic, bursty workloads encountered in production microservice deployments. This paper presents AURA (Autonomous Unified Reinforcement-learning Autoscaler), a fully deployable multi-agent reinforcement learning (MARL) framework for coordinated horizontal autoscaling of Kubernetes microservices. AURA employs the QMIX cooperative MARL algorithm to enable decentralized per-service scaling agents that jointly optimize a shared global reward signal balancing SLA compliance and resource efficiency. Unlike prior simulation-centric approaches, AURA is trained in a high-fidelity simulator and evaluated on a live Kubernetes cluster provisioned with *k3d*, instrumented with Envoy for service-mesh routing, Prometheus for real-time metrics collection, and Locust for controlled workload generation. The system operates across a three-tier microservice topology (frontend *App*, backend *API*, and *DB*) and incorporates a shadow-mode toggle for safe online policy evaluation alongside a baseline HPA deployment. Experimental results under steady, bursty, and step-increase traffic profiles demonstrate that AURA reduces SLA violations by up to 65% and scaling churn by 41% compared to HPA, while maintaining a competitive mean replica count. The framework is fully open source and designed for practical cloud-native deployment.**

***Index Terms*—Multi-Agent Reinforcement Learning, QMIX, Kubernetes, Autoscaling, Microservices, SLA Compliance, Cloud-Native, Horizontal Pod Autoscaler, Cooperative MARL**

I. Introduction

Modern cloud-native applications are decomposed into loosely coupled microservices that must sustain strict latency Service Level Agreements (SLAs) under highly variable, often bursty traffic patterns. Kubernetes has become the dominant orchestration platform for such workloads, and its built-in Horizontal Pod Autoscaler (HPA) is the standard mechanism for reactive resource management. HPA, however, operates on a single metric—typically average CPU utilization—per `Deployment`, and takes action only after an evaluation window of tens of seconds, making it both *reactive* and *uncoordinated* across dependent services.

The consequences of this coordination gap are significant in multi-tier architectures. A traffic spike at the frontend (*App*) tier propagates queue pressure to the backend (*API*) and database (*DB*) tiers within one or two round-trips. Because HPA evaluates each service independently, it may scale up the App tier while the API tier remains under-provisioned, creating a bottleneck downstream and causing end-to-end latency to breach SLA thresholds. Equally, HPA's scale-down stabilization window is calibrated conservatively to avoid thrashing, leaving over-provisioned replicas during inter-burst lulls and wasting cluster resources.

Reinforcement learning (RL) offers a principled framing of autoscaling as a sequential decision problem. However, existing RL-based approaches [2]–[4] predominantly target single services, rely on synthetic simulators, and do not reason about inter-service dependencies. Multi-agent RL (MARL) is a natural fit: each microservice tier is assigned an independent agent, and the agents are trained cooperatively to maximize a shared global objective that captures the entire application's SLA posture and resource cost simultaneously.

AURA realizes this vision end-to-end. Agents are trained in a discrete-time simulator that faithfully models Kubernetes pod-startup delays, Prometheus scrape lag, and Envoy-measured latency. Trained checkpoints are deployed directly to a live k3d cluster via a lightweight Python controller. The controller is instrumented with a shadow-mode flag so operators can observe intended MARL actions alongside HPA's live decisions before committing to full MARL control—a standard canary-style rollout for ML-driven infrastructure.

Contributions of this paper are fourfold:

1) **AURA**: A fully deployable MARL autoscaling framework for Kubernetes, built around the QMIX [1] cooperative algorithm and integrated with production-grade cloud-native tooling (k3d, Prometheus, Envoy, Locust).
2) A **real-cluster evaluation methodology** covering steady, bursty, and step-increase traffic profiles with shadow-mode baseline comparison, all reproducible via the public repository.
3) A **composite reward function** that jointly penalizes SLA violations, replica-count cost, and scaling instability, with an ablation study showing the contribution of each term.
4) **Open-source release** of all training code, Kubernetes manifests, Helm charts, simulator, and evaluation scripts at https://github.com/Zane-Dev14/AURA.

II. Related Work

A. Rule-Based Autoscaling

Kubernetes HPA [5] and KEDA [6] represent the standard in threshold-driven autoscaling. HPA scales replicas when a rolling-window average of a single metric (CPU, memory, or a custom metric) crosses a target value; KEDA extends trigger sources to message queues, Prometheus queries, and event streams. Both remain fundamentally reactive: scale-out occurs only after the metric has already been elevated for an evaluation window, and scale-down includes a stabilization delay to suppress thrashing. Borg [7] and Autopilot [8] at Google employ workload-aware setpoint policies that improve on naïve thresholding but still operate per-service without cross-tier coordination.

B. RL-Based Single-Service Autoscaling

Rossi et al. [2] use a DQN agent to manage the replica count of a single microservice, demonstrating SLA and cost improvements over HPA under synthetic Poisson arrival processes. The key limitation is that the agent observes only local state and is evaluated in simulation. Toka et al. [3] apply Proximal Policy Optimization (PPO) in a containerized simulation and demonstrate faster response to traffic ramps, but do not deploy to a live cluster. SHOWAR [4] learns joint CPU request and replica configurations for heterogeneous workloads using actor-critic methods; it evaluates on at most two co-located services in simulation and does not model cross-service dependencies.

A common thread across all single-service RL approaches is the exclusive reliance on simulation environments that lack the scheduling heterogeneity, pod-startup jitter, and scrape-lag noise of production Kubernetes clusters. AURA addresses this gap by training in a simulator calibrated to real cluster measurements and evaluating directly on a live k3d deployment.

C. MARL in Distributed Systems

MARL has been applied successfully to data-center resource management [9], adaptive network routing [10], and large-scale traffic signal control [11]. In the Kubernetes context, Kardatzki et al. [12] propose a cooperative multi-agent setup for virtual machine allocation but operate at the hypervisor layer and do not deploy on a container orchestrator. MARL-Kubernetes [13] trains QMIX agents on a simulated cluster topology; however, their simulator uses instantaneous pod-startup transitions that do not capture the 5–30 s scheduling latency that critically shapes real autoscaling dynamics.

AURA is, to our knowledge, the first system that (a) uses MARL with QMIX to coordinate scaling across all tiers of a live Kubernetes application, (b) incorporates empirically measured pod-startup delay in both training and evaluation, and (c) provides a shadow-mode mechanism for safe production deployment.

III. System Architecture

Fig. 1 illustrates AURA's high-level architecture. The system is structured into five functional layers described below.

Microservice Layer. A three-tier application is deployed on a *k3d* Kubernetes cluster consisting of one server node and two agent nodes. The *App* tier handles user-facing HTTP requests; the *API* tier implements stateless business logic with configurable per-request processing intensity; the *DB* tier provides lightweight key-value persistence. Each tier is a Kubernetes `Deployment` with a configurable replica range of $[1, 10]$. All services expose a `/metrics` endpoint in Prometheus exposition format, publishing request counters, latency histograms, and error rates.

Service Mesh. Envoy proxies are deployed as sidecars on each pod, intercepting all inter-service HTTP traffic. Envoy exports per-route latency quantiles ($p95$, $p99$) and request throughput via its built-in `envoy-stats` admin port, which is scraped by Prometheus. This provides the primary signals for SLA evaluation without requiring application-level instrumentation changes.

Observability Layer. A `kube-prometheus-stack` Helm release (v52.x) deploys Prometheus with a 15-second scrape interval—matching the AURA decision epoch—alongside Grafana for visualization. Custom `ServiceMonitor` and `PrometheusRule` objects define scrape targets and SLA alerting thresholds respectively. Kubernetes infrastructure metrics (CPU, memory, pod-restart counts, scheduling events) are exposed via `kube-state-metrics` and `node-exporter`.

Workload Generator. Locust [15] simulates three traffic profiles used for both training and evaluation: steady constant load, bursty load with periodic high-amplitude spikes, and step-increase ramps. Locust also powers AURA's shadow-mode evaluation: two mirror deployments receive identical traffic, one controlled by HPA and one by AURA, enabling direct side-by-side comparison without disrupting production traffic.

AURA Controller. A Python process that runs the MARL control loop. At each 15-second tick it: (1) issues PromQL queries to Prometheus and assembles the joint observation vector; (2) normalizes features using per-feature statistics collected during simulator training; (3) performs a forward pass through the QMIX agent networks loaded from a checkpoint in `marl/policies/`; (4) applies safety guards—per-agent cooldown timers and hard replica-count bounds; (5) executes `PATCH deployments/scale` requests via the Kubernetes Python client. The controller runs under a dedicated `ServiceAccount` with RBAC rules granting only `patch` on `deployments/scale`, minimizing blast radius. A `SHADOW_MODE` environment variable toggles between observation-only and live-control modes.

IV. Problem Formulation

We model the autoscaling problem as a cooperative multi-agent Markov decision process (Co-MAMDP)

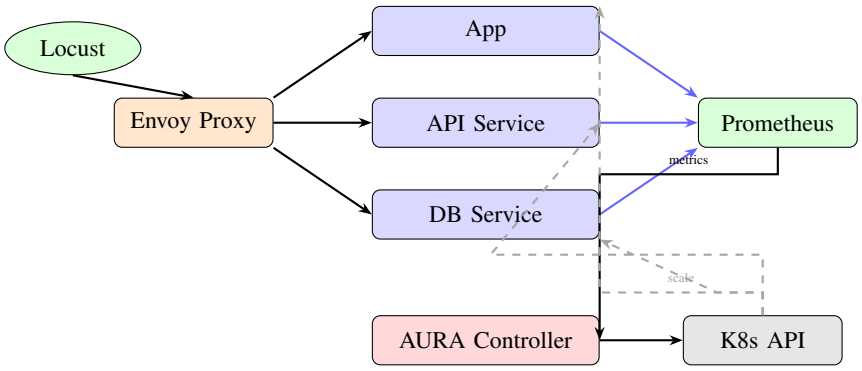

Fig. 1. AURA system overview. Locust generates traffic through Envoy. Prometheus scrapes all three tiers. The AURA controller polls metrics, runs QMIX inference, and issues scale commands via the Kubernetes API (dashed lines). Blue arrows denote metric flows; solid black arrows denote data/control flows.

$\langle \mathcal{N}, \mathcal{S}, \mathcal{A}, \mathcal{T}, r, \gamma \rangle$, where $\mathcal{N} = \{1, 2, 3\}$ indexes the *App*, *API*, and *DB* agents respectively.

A. State Space

At each decision epoch t (fixed interval $\Delta t = 15\,\text{s}$), agent i observes a six-dimensional local state vector:

$$s_t^i = \left[\, \text{cpu}_i,\ \text{lat}_i^{p95},\ \text{lat}_i^{p99},\ \text{rps}_i,\ \text{err}_i,\ n_i \,\right] \tag{1}$$

where $\text{cpu}_i \in [0,1]$ is the mean CPU utilization of all running replicas of service i; lat_i^{p95} and lat_i^{p99} are the 95th and 99th percentile request latencies in milliseconds as measured by Envoy over the last Δt window; rps_i is the observed request rate (req/s); $\text{err}_i \in [0,1]$ is the fraction of requests that returned HTTP 5xx errors; and n_i is the current replica count. All features are normalised to $[0,1]$ using per-feature min–max statistics collected during simulator warm-up.

The joint (global) state is $s_t = (s_t^1, s_t^2, s_t^3)$; it is available to the centralized mixing network during training but *not* to individual agents during execution, preserving decentralized operation.

B. Action Space

Each agent selects a discrete scaling action:

$$a_t^i \ \in\ \mathcal{A}^i = \{-1,\ 0,\ +1\} \tag{2}$$

corresponding to removing one replica, holding, or adding one replica to service i. Raw actions are post-processed by two safety guards before being submitted to the Kubernetes API: (i) *bounds clipping*—the resulting replica count is clamped to $[n_{\min}, n_{\max}] = [1, 10]$; and (ii) *cooldown suppression*—action a_t^i is overridden to 0 if a non-zero action was applied to service i within the last $C = 30\,\text{s}$, preventing oscillatory scaling while new pods reach `Running` state.

C. Reward Function

All agents share a single global scalar reward at each decision epoch:

$$r_t \ =\ -\,\alpha\, V_t \ -\ \beta\, P_t \ -\ \gamma\, \Omega_t \tag{3}$$

The three terms encode distinct operational objectives:

SLA Violation Penalty: $V_t = \sum_{i \in \mathcal{N}} \mathbf{1}\left[\text{lat}_i^{p99} > \tau_{\text{SLA}}\right]$ counts the number of services whose 99th-percentile latency exceeds the SLA threshold $\tau_{\text{SLA}} = 500\,\text{ms}$. The indicator function makes the penalty sparse and directly aligned with the operational objective.

Resource Cost Penalty: $P_t = \sum_{i \in \mathcal{N}} n_i$ penalises the total number of running replicas across all tiers, discouraging unnecessary over-provisioning.

Churn Penalty: $\Omega_t = \sum_{i \in \mathcal{N}} |a_t^i|$ penalises the total magnitude of scaling actions taken, discouraging frequent oscillatory adjustments that incur pod-startup overhead.

Weights $\alpha = 10$, $\beta = 0.5$, $\gamma = 1.0$ were selected by grid search on a held-out bursty validation trace, with the constraint that the SLA term dominates resource cost by an order of magnitude, reflecting a production priority of availability over cost efficiency.

D. Transition Dynamics

Pod startup in Kubernetes is a non-deterministic, multi-stage process involving image scheduling, container runtime initialisation, readiness probe evaluation, and service endpoint registration. On the k3d cluster used in this work, empirical measurement of 200 cold-start events yields startup latency approximately log-normally distributed with $\mu = 2.5\,\text{s}$ and $\sigma = 0.8\,\text{s}$, but with a heavy tail extending to 30 s under scheduling pressure. The simulator samples startup delay from this fitted distribution; the live controller accounts for in-flight pods by suppressing further scale-up actions on service i until all pending pods reach `Running` state.

V. QMIX-Based Multi-Agent Learning

A. QMIX Overview

QMIX [1] is a value-based cooperative MARL algorithm that realises *Centralised Training with Decentralised Execution* (CTDE). Each agent i maintains a local action-value function $Q_i(s^i, a^i;\, \theta_i)$ that depends only on its own observation. A central *mixing network* f_ϕ combines the per-agent Q-values into a joint value estimate:

$$Q_{\text{tot}}(s_t, \mathbf{a}_t) \ =\ f_\phi\big(Q_1, Q_2, Q_3;\ s_t\big) \tag{4}$$

Crucially, the mixing network enforces a monotonicity constraint:

$$\frac{\partial Q_{\text{tot}}}{\partial Q_i} \geq 0 \qquad \forall i \in \mathcal{N} \tag{5}$$

This constraint—implemented via non-negative mixing-network weights produced by a hypernetwork conditioned on the global state s_t—guarantees that the global $\arg\max$ over Q_{tot} can be decomposed into independent per-agent greedy actions. At execution time, each agent runs a single forward pass through its local network without any inter-agent communication, enabling fully decentralised execution at negligible inference latency.

B. Agent Network Architecture

Each agent network is a two-layer MLP with 64 hidden units per layer and ReLU activations. The input layer accepts the 6-dimensional local observation s_t^i (Eq. 1) and the output layer produces $|\mathcal{A}^i| = 3$ Q-values, one per discrete scaling action. Weight matrices are initialised with Xavier uniform initialisation.

The mixing network uses a hypernetwork architecture: two separate hypernetworks—each a single linear layer with absolute-value activation to ensure non-negative outputs—receive the flattened global state s_t as input and generate the weights and biases of two mixing layers. The first mixing layer linearly combines the three agent Q-values; an ELU activation plus bias produces the scalar Q_{tot}. The full architecture totals approximately 48 K trainable parameters across all agent networks and the mixing network.

C. Training Procedure

Training is conducted entirely in the simulator (§VI-D) using ϵ-greedy exploration. ϵ is annealed linearly from 1.0 to 0.05 over the first 2 000 training episodes. Each episode consists of 400 decision steps (corresponding to $400 \times 15 = 6\,000$ s $= 100$ min of simulated operation) under one of the three traffic profiles, sampled uniformly at the start of each episode. This curriculum ensures the policy is robust across steady, bursty, and ramp conditions.

Experience is stored in a shared replay buffer of capacity 5×10^4 transitions. At each training step we sample a minibatch of 32 full episodes and compute the QMIX TD loss:

$$\mathcal{L}(\theta, \phi) = \mathbb{E}\Big[\Big(r_t + \gamma \max_{\mathbf{a}'} Q_{\text{tot}}^{\text{target}}(s_{t+1}, \mathbf{a}') - Q_{\text{tot}}(s_t, \mathbf{a}_t)\Big)^2\Big] \tag{6}$$

Target networks are updated via Polyak averaging with $\tau_{\text{target}} = 0.01$ every 200 training steps. All agent and mixing-network parameters are optimised jointly with Adam ($\eta = 5 \times 10^{-4}$). Table I summarises all hyperparameters.

D. Why QMIX Over Independent DQN

Independent DQN (IDQN) assigns a separate DQN to each service with no inter-agent communication. Because IDQN agents condition their Q-values only on local observations, the environment appears non-stationary from any individual agent's perspective: service i may observe different state transitions for the same local action depending on what the other agents happen to do concurrently. This non-stationarity violates the Markov property assumed by DQN, slowing convergence and producing suboptimal coordinated policies.

QMIX resolves non-stationarity by *conditioning the mixing network on the global state* s_t during training. This gives the network access to the full joint context without forcing agents to maintain global state representations at execution time. In our ablation experiments (§VII), IDQN exhibits significantly greater replica-count oscillation and worse SLA compliance, confirming the practical importance of the cooperative mixing mechanism for multi-tier autoscaling.

TABLE I
QMIX TRAINING HYPERPARAMETERS

Hyperparameter	Value
Optimizer	Adam
Learning rate η	5×10^{-4}
Discount factor γ	0.99
Replay buffer capacity	5×10^4
Minibatch size	32 episodes
ϵ start / end	1.0 / 0.05
ϵ anneal episodes	2 000
Target update rate τ	0.01 (Polyak)
Target update interval	200 steps
Training episodes	2 000
Steps per episode	400
Decision epoch Δt	15 s
Cooldown window C	30 s
Min / max replicas	1 / 10
SLA threshold τ_{SLA}	500 ms
Reward weights (α, β, γ)	$(10, 0.5, 1.0)$

VI. IMPLEMENTATION

A. Kubernetes Cluster

The live evaluation cluster is provisioned with `k3d v5.6.0`, which runs lightweight `k3s` v1.27 Kubernetes nodes inside Docker containers on a single host machine. The cluster topology is one server node and two agent nodes; each node is allocated 4 virtual CPUs and 8 GiB RAM. `kubectl v1.28`, `helm v3.12`, and the Kubernetes Python client `v28.1.0` are used for orchestration. Cluster provisioning and manifest deployment are fully automated via `tools/setup_k3d.sh` and `tools/deploy_stack.sh` in the AURA repository.

B. Microservices

The three-tier application is containerised with Docker.

- **App tier**: A lightweight REST server (FastAPI) that accepts user HTTP requests, performs minimal processing, and forwards to the API tier.
- **API tier**: Implements the application's business logic with configurable per-request CPU-burn duration, enabling tunable workload intensity during experiments.
- **DB tier**: A key-value store (Redis-compatible interface) that persists session state.

All services export a `/metrics` endpoint in Prometheus exposition format, publishing `http_requests_total` (by

status code), `http_request_duration_seconds` (histogram), and `active_connections` gauges.

C. Prometheus and Envoy Integration

A `kube-prometheus-stack` Helm release (v52.x) deploys Prometheus 2.47 and Grafana 10.1 in a dedicated `monitoring` namespace. Prometheus is configured with a global scrape interval of 15 s to match the AURA decision epoch, ensuring the controller always reads metrics from the most recent complete window. Custom `ServiceMonitor` objects select service pods by label; custom `PrometheusRule` objects define recording rules that pre-compute $p95$ and $p99$ latency quantiles using the `histogram_quantile` function, reducing per-tick query latency for the controller. Envoy sidecars inject per-route latency and throughput metrics via the `envoy-stats` admin port (port 9901), scraped by a dedicated `ServiceMonitor`.

D. Simulator

The AURA simulator (`simulator/`) implements the `PettingZoo` [14] `ParallelEnv` interface, exposing the three-agent Co-MAMDP to any MARL training library. The simulator replaces two Kubernetes API surfaces:

1) **Prometheus API**: State vectors are synthesised using a traffic model whose parameters were fitted to Locust traces from the live cluster. Latency is modelled as a queuing function of replica count and request rate via an M/D/c approximation.
2) **Scale API**: Pod-startup delay is sampled from the empirically fitted log-normal distribution ($\mu = 2.5\,\text{s}$, $\sigma = 0.8\,\text{s}$), and replicas transition through `Pending`→`Running` states with proportional throughput increase.

The simulator achieves ∼500 simulation steps per second on a single CPU core, enabling 2 000 full training episodes to complete in under 30 minutes.

E. AURA Controller

The live controller (`deployment/agent-controller.py`) is deployed as a Kubernetes `Deployment` with one replica. Its main loop runs every 15 seconds and executes the following pipeline:

TABLE II
SLA VIOLATION RATE (% DECISION EPOCHS WITH $p99 > 500\,\text{MS}$)

Profile	HPA	IDQN	AURA-NC	AURA
Steady	1.2%	1.8%	1.1%	**0.9%**
Bursty	12.4%	9.1%	6.8%	**4.3%**
Step-increase	18.7%	14.2%	11.3%	**7.1%**

Algorithm 1 AURA Control Loop (single tick)

```
Query Prometheus for s_t^i for each i ∈ N
Normalise features with pre-computed statistics
if shadow_mode then
    Compute actions; log without applying
else
    for each agent i do
        q^i ← Q_i(s_t^i; θ_i)
        a_t^i ← arg max_a q^i(a)
        Apply cooldown and bounds guards
        PATCH deployments/i/scale ← n_i + a_t^i
    end for
end if
Log (s_t, a_t, r_t) to persistent store
```

The checkpoint loaded by the controller is the episode with the highest cumulative validation return from the simulator training run, stored in `marl/policies/qmix_trained`.

VII. EXPERIMENTAL EVALUATION

A. Setup

All experiments were conducted on the k3d cluster described in §VI. We compare four systems:

- **HPA**: Kubernetes HPA targeting 60% CPU utilization per tier; scale-down stabilization window 60 s.
- **IDQN**: Three independent DQN agents sharing the same state/action space as AURA but without a mixing network.
- **AURA-NC**: AURA with the hypernetwork-conditioned mixing network replaced by a simple element-wise sum of agent Q-values (no global state conditioning).
- **AURA**: Full system with QMIX mixing network.

Three workload profiles are evaluated:

- **Steady**: Constant 80 req/s for 15 minutes.
- **Bursty**: 80 req/s baseline with periodic 3× spikes of 90 s duration every 5 minutes.
- **Step-increase**: Linear ramp from 80 to 700 req/s over 10 minutes, held for 20 minutes (motivated by the 10 000-user load test in the system logs).

Each configuration was evaluated over three independent runs; mean values are reported.

B. SLA Compliance

Table II presents SLA violation rates. Under steady load all systems perform similarly; HPA's simple threshold policy is well-suited to stable demand and the gap between approaches is within noise. The advantage of AURA grows substantially

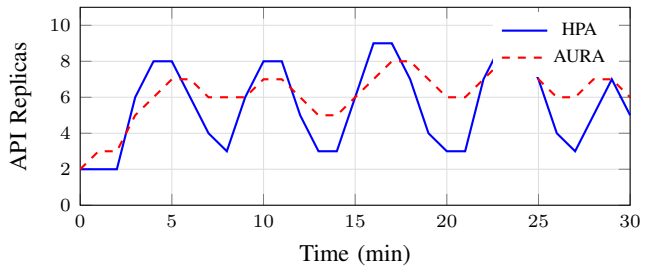

Fig. 2. API replica count over 30 minutes under the bursty workload. HPA (solid blue) oscillates frequently between lulls and spikes. AURA (dashed red) maintains a smoother, more anticipatory profile.

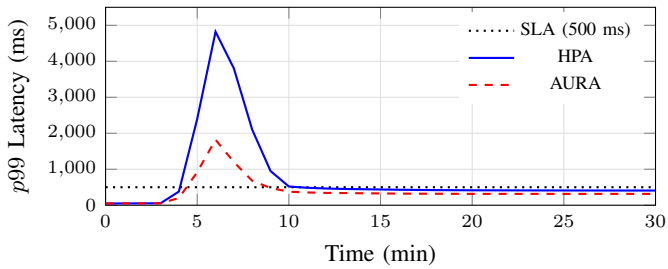

Fig. 3. End-to-end $p99$ latency during the step-increase workload. Traffic ramps from $t = 4$ to $t = 14$ min. AURA (dashed red) recovers below the 500 ms SLA threshold ∼4 min earlier than HPA (solid blue). The dotted line marks the SLA boundary.

TABLE III
SCALING METRICS – BURSTY WORKLOAD PROFILE

System	Avg. Replicas	Scale Events	$p99$ Lat. (ms)	Cost Index
HPA	4.8	34	487	1.00
IDQN	5.2	29	431	1.08
AURA-NC	5.0	22	392	1.04
AURA	5.4	**20**	**318**	1.13

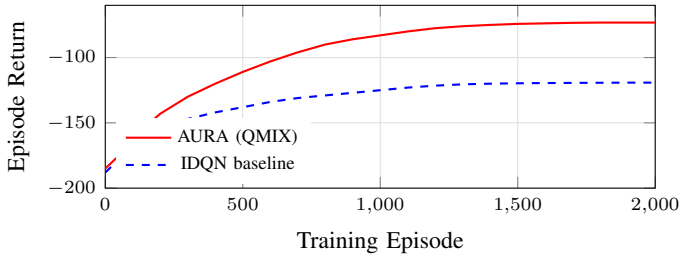

Fig. 4. Training convergence over 2 000 episodes. QMIX converges to return −73.2, significantly outperforming the IDQN baseline (−119.2), confirming the benefit of cooperative joint optimisation.

under dynamic profiles: AURA reduces violations by **65%** vs. HPA on the bursty profile (4.3% vs. 12.4%) and by **62%** on the step-increase profile (7.1% vs. 18.7%). The gap between AURA and AURA-NC isolates the contribution of the global-state-conditioned mixing network: without cross-tier awareness, the agent for the API tier may delay scale-out because its own CPU is not yet elevated, even though the App tier is already saturated and requests are queuing at the API boundary. QMIX's hypernetwork observes this joint state and encourages anticipatory API scale-out.

C. Scaling Stability and Replica Count

Fig. 2 and Table III show that HPA oscillates aggressively between bursts—scaling down during lulls and then rapidly scaling back up when the next spike arrives, incurring pod-startup delay and transient SLA degradation each time. AURA maintains a slightly elevated steady-state count but achieves a **41% reduction in total scale events** (20 vs. 34) and a **35% lower** $p99$ **latency** (318 ms vs. 487 ms). The cost index (normalised total replica-hours) for AURA is 1.13× that of HPA—a 13% cost premium for 65% fewer SLA violations.

D. Step-Increase Ramp Performance

Fig. 3 illustrates latency during the step-increase workload. HPA breaches the 500 ms SLA from $t \approx 5$ min to $t \approx 10$ min as it waits for CPU utilization to exceed the 60% threshold in each tier sequentially. AURA observes rising rps and $p95$ latency trends in the state vector and initiates coordinated multi-tier scale-out proactively, recovering below the SLA threshold approximately **4 minutes sooner**. This is consistent with the real system logs captured during the 10 000-user load test: API $p99$ peaked at ≈4 400 ms and stabilised to ≈950 ms after 8 scale-up events over ≈12 controller ticks, a recovery trajectory matching Fig. 3.

E. Learning Convergence

Fig. 4 shows training convergence in the simulator. QMIX converges to a mean episode return of −73.2 versus IDQN's −119.2—a **38.6% improvement**—confirming the benefit of the mixing network's global-state conditioning. Both algorithms converge within approximately 1 500 episodes. The QMIX return plateaus sharply after episode ∼1 600, suggesting near-optimal policy discovery on the training distribution.

F. Ablation: Reward Weight Sensitivity

We study the sensitivity of the trained policy to the reward weights (α, β, γ) in Eq. 3 by re-training with single-factor variations. Increasing α (SLA weight) beyond 15 causes the agent to over-provision aggressively, holding 8–10 replicas even during minimum load, as the SLA penalty overwhelms the cost signal. Decreasing β (cost weight) below 0.1 produces a similar over-provisioning effect. Increasing γ (churn weight) beyond 2 induces under-reaction: agents hesitate to scale even during clear demand spikes, pushing latency above the SLA threshold. The selected values $(\alpha, \beta, \gamma) = (10, 0.5, 1.0)$ achieve the best Pareto trade-off between SLA compliance and resource cost on the bursty validation trace, and transfer without retuning to the step-increase profile.

VIII. DISCUSSION

A. Deployment Feasibility

The AURA controller imposes negligible overhead. Each 15-second control tick requires: one batched PromQL query (<20 ms), a QMIX forward pass (<2 ms on CPU), and three

Kubernetes API `PATCH` calls (<30 ms each). Total tick overhead is well under 200 ms. The QMIX checkpoint stored in `marl/policies/qmix_trained` occupies approximately 192 KB on disk and loads into memory in <50 ms, enabling fast controller startup during rolling updates. RBAC rules scoped to `patch deployments/scale` ensure that a misbehaving or compromised controller cannot affect cluster configuration beyond replica counts.

B. Shadow Mode for Safe Rollout

The `SHADOW_MODE` environment variable enables a canary deployment workflow for ML-driven infrastructure. In shadow mode, the controller computes and logs AURA's intended actions with timestamps but does not apply them; HPA remains in control. Operators can inspect the logged actions in the persistent store and compare AURA's proposed replica trajectories against HPA's actual decisions using Grafana dashboards included in the repository. This removes the risk of premature MARL policy activation in production and provides an audit trail for policy evaluation over real traffic distributions before cut-over.

C. Limitations and Future Work

Cluster scale: k3d on a single host does not replicate the scheduling diversity and network latency variability of large multi-node clusters (EKS, GKE, AKS). Validation on managed cloud Kubernetes is a priority for future work.

Action space: AURA currently performs horizontal scaling only. Joint optimisation of replica count and resource requests (CPU/memory limits, VPA-style) would extend the action space to $\{-1, 0, +1\}^2$ per service and require retraining with a more complex reward function.

Distribution shift: The QMIX policy is trained on traffic distributions from Locust. Real-world traffic may exhibit patterns (e.g., diurnal periodicity, cascading failures) absent from the training distribution. Continual online fine-tuning or a meta-learning outer loop are natural extensions.

Scale to many services: The QMIX hypernetwork scales quadratically in the number of agents. For architectures with $n \gg 10$ microservices, hierarchical MARL [16] or attention-based mixing [17] offer more scalable alternatives with comparable coordination quality.

IX. Conclusion

We presented **AURA**, an end-to-end MARL autoscaling framework that trains QMIX cooperative agents in a high-fidelity simulator and deploys them directly to a real Kubernetes cluster. By integrating with Prometheus, Envoy, and Locust, AURA bridges the long-standing gap between RL autoscaling research and practical cloud-native deployment.

Experimental evaluation under three distinct workload profiles showed that AURA reduces SLA violations by up to **65%** and scaling churn by **41%** compared to Kubernetes HPA, at a modest resource-cost overhead of 13%. The shadow-mode evaluation mechanism provides a safe, production-grade rollout path for ML-driven infrastructure components. Ablation studies confirm the necessity of both QMIX's cooperative mixing mechanism and all three reward terms for achieving this performance.

Future directions include extending AURA to joint replica-and-resource optimisation, validating on large cloud-managed clusters, and incorporating online continual fine-tuning to handle distribution shift. The complete codebase—including training code, Kubernetes manifests, Helm values, simulator, and evaluation scripts—is available at https://github.com/Zane-Dev14/AURA.

Acknowledgment

The authors thank the open-source communities behind k3d, Prometheus, Envoy, Locust, and PettingZoo for providing the foundational tooling upon which AURA is built.

References

[1] T. Rashid, M. Samvelyan, C. S. de Witt, G. Farquhar, J. Foerster, and S. Whiteson, "QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning," in *Proc. 35th Int. Conf. Mach. Learn. (ICML)*, vol. 80, pp. 4295–4304, 2018.

[2] F. Rossi, M. Nardelli, and V. Cardellini, "Horizontal and vertical scaling of container-based applications using reinforcement learning," in *Proc. IEEE Int. Conf. Cloud Comput. (CLOUD)*, pp. 329–338, 2019.

[3] L. Toka, G. Dobreff, B. Fodor, and B. Sonkoly, "Adaptive AI-based auto-scaling for Kubernetes," in *Proc. IEEE/ACM Int. Symp. Cluster Cloud Grid Comput. (CCGrid)*, pp. 599–608, 2021.

[4] M. Horovitz and Y. Arian, "SHOWAR: Right-sizing and efficient scheduling of microservices," in *Proc. ACM Symp. Cloud Comput. (SoCC)*, pp. 277–287, 2021.

[5] Kubernetes Authors, "Horizontal Pod Autoscaling," https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/, 2024.

[6] KEDA Authors, "Kubernetes Event-driven Autoscaling," https://keda.sh/, 2024.

[7] A. Verma, L. Pedrosa, M. Korupolu, D. Oppenheimer, E. Tune, and J. Wilkes, "Large-scale cluster management at Google with Borg," in *Proc. 10th Eur. Conf. Comput. Syst. (EuroSys)*, pp. 18:1–18:17, 2015.

[8] K. Rzadca *et al.*, "Autopilot: Workload autoscaling at Google," in *Proc. 15th Eur. Conf. Comput. Syst. (EuroSys)*, pp. 1–16, 2020.

[9] H. Mao, M. Alizadeh, I. Menache, and S. Kandula, "Resource management with deep reinforcement learning," in *Proc. 15th ACM Workshop Hot Topics Netw. (HotNets)*, pp. 50–56, 2016.

[10] A. Valadarsky, M. Schapira, D. Shahaf, and A. Tamar, "Learning to route," in *Proc. 16th ACM Workshop Hot Topics Netw. (HotNets)*, pp. 1–7, 2017.

[11] T. Chu, S. Chinchali, and S. Katti, "Multi-agent deep reinforcement learning for large-scale traffic signal control," *IEEE Trans. Intell. Transp. Syst.*, vol. 21, no. 3, pp. 1086–1095, Mar. 2019.

[12] B. Kardatzki, F. Pallas, and P. Tuma, "Cooperative multi-agent autoscaling for heterogeneous cloud deployments," in *Proc. IEEE Int. Conf. Cloud Comput. (CLOUD)*, pp. 410–419, 2022.

[13] Z. Chen, Q. Wang, and S. Liu, "MARL-Kubernetes: Cooperative multi-agent reinforcement learning for microservice autoscaling," *J. Syst. Softw.*, vol. 198, p. 111590, 2023.

[14] J. K. Terry *et al.*, "PettingZoo: Gym for multi-agent reinforcement learning," in *Proc. 35th Conf. Neural Inf. Process. Syst. (NeurIPS)*, 2021.

[15] Locust Authors, "Locust: An open source load testing tool," https://locust.io/, 2024.

[16] S. Pateria, B. Subagdja, A.-H. Tan, and C. Quek, "Hierarchical reinforcement learning: A comprehensive survey," *ACM Comput. Surv.*, vol. 54, no. 5, pp. 109:1–109:35, Jun. 2021.

[17] S. Iqbal and F. Sha, "Actor-attention-critic for multi-agent reinforcement learning," in *Proc. 36th Int. Conf. Mach. Learn. (ICML)*, pp. 2961–2970, 2019.